

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



7일차

02 Sensing Zone – AI Model 적용 (Yolo+UFLD)

Telechips  **TOPST**

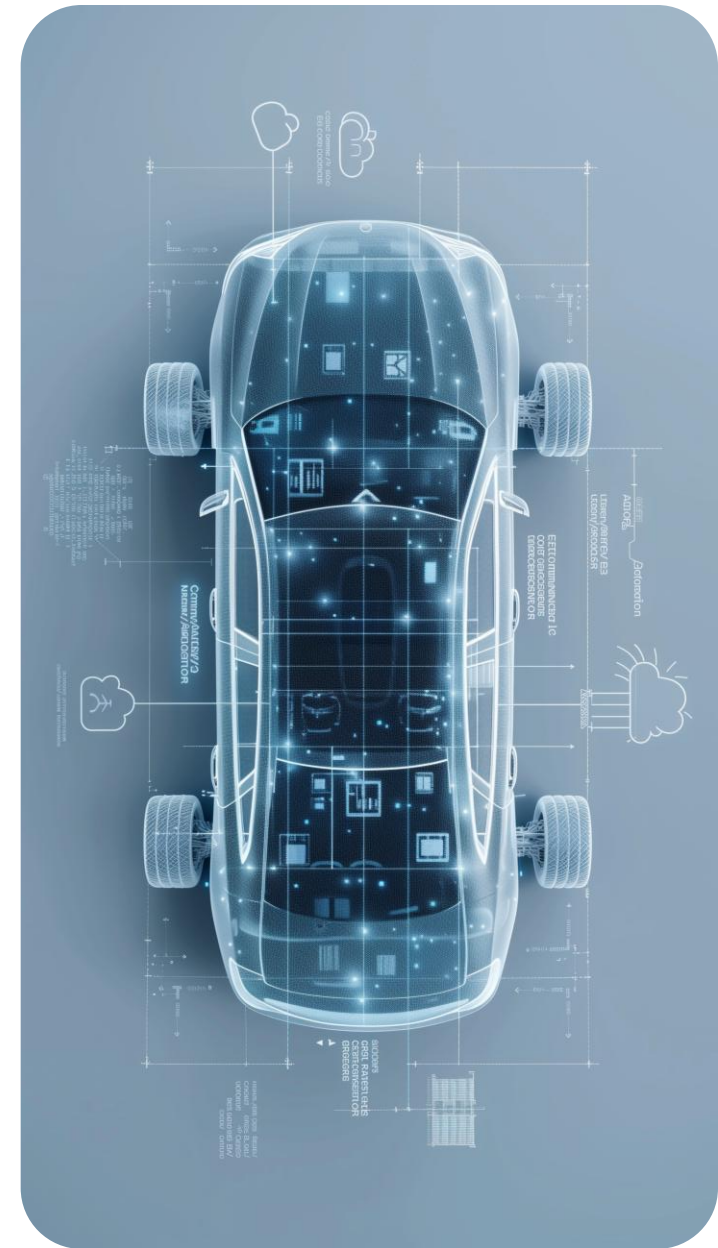


목차

Table of Contents

TC-NN-APP 소스 수정

01



TC-NN-APP 소스 수정

- tcnnapp에서 차선 인식을 위한 후처리 및 시각화 소스 추가



TC-NN-APP 소스 수정

- tcnnapp에서 차선 인식을 위한 후처리 및 시각화 소스 추가

```
$ cd ~/Work/ai-g-yocto/build/ai-g-topst/tmp/work/cortexa53-telechips-  
linux/tc-nn-app/1.0.0-r0/git
```

```
$ ls
```

```
ead@TC-D-20-0546:~/topst-sdk/build/ai-g-topst/tmp/work/cortexa53-telechips-linux/tc-nn-app/1.0.0-r0/git$ ls  
CMakeLists.txt  common  include  src
```

```
$ ls src/
```

```
ead@TC-D-20-0546:~/topst-sdk/build/ai-g-topst/tmp/work/cortexa53-telechips-linux/tc-nn-app/1.0.0-r0/git$ ls src/  
NnAppMain.c  NnDebug.h  NnNeuralNetwork.c  NnPerf.h  NnRtpm.h  
NnAppMain.h  NnMemory.c  NnNeuralNetwork.h  NnProtocolManager.c  NnSignalHandler.c  
NnDebug.c  NnMemory.h  NnPerf.c  NnRtpm.c  NnSignalHandler.h
```

TC-NN-APP 소스 수정

- tcnnapp에서 차선 인식을 위한 후처리 및 시각화 소스 추가

```
$ vi include/NnType.h
```

상단에 차선 인식을 위한 구조체 추가

```
typedef struct {  
    float loc[72];  
} custom_lane_t;
```

네트워크 정보를 저장하는 _network_context 구조체 lane 구조체 선언

```
typedef struct _network_context  
{  
    char *networkPath;  
    ....  
    enlight_postproc_t type;  
    enlight_batch_cls_t resultCls;  
    enlight_objc_t resultObjc;  
    custom_lane_t lane_data;  
} network_context_t;
```

해당 내용 추가

TC-NN-APP 소스 수정

- tcnnapp에서 차선 인식을 위한 후처리 및 시각화 소스 추가

```
$ vi src/NnNeuralNetwork.c
```

네트워크 출력 정보를 차선 구조체에 저장하도록 NnRunInference 함수 소스 수정

```
void NnRunInference(network_context_t *pNeuralNetwork, npu_run_mode_t npuRunMode)
{
    npu_err_bits_t err_status;
    ...
    else if (pNeuralNetwork->type == TELECHIPS_NPU_POST_CUSTOM)
    {
        network_run_postprocess(pNeuralNetwork->networkHandle, pNeuralNetwork->outputBuf, &(pNeuralNetwork->lane_data));
    }
    else
    {
        // TBD
    }
}
```

해당 내용으로 수정

TC-NN-APP 소스 수정

- tcnnapp에서 차선 인식을 위한 후처리 및 시각화 소스 추가

```
$ vi src/NnAppMain.c
```

차선 좌표를 스케일링하고 시각화를 위한 NnDrawResult 함수 내부에 기능 추가

```
static void NnDrawResult(app_context_t *pContext)
{
    ...
    else if(pNeuralNetwork->type ==
    TELECHIPS_NPU_POST_CUSTOM)
    {
        float *lane_loc = pNeuralNetwork->lane_data.loc;
        Color_t yellow = RGB(255, 255, 0);
        int num_rows = 18;
        int num_lanes = 4;
        const int griding_num = 200;
        const float col_sample_w = (800.0f - 1) /
        (griding_num - 1);
        const int row_anchor[18] = {
            121, 131, 141, 150, 160, 170, 180, 189, 199,
            209, 219, 228, 238, 248, 258, 267, 277, 287
        };
        Point_t lanePoints[72];

        int pointIdx = 0;
```

```
for (int j = 0; j < num_rows; j++) {
    for (int k = 0; k < num_lanes; k++) {
        float x_norm = lane_loc[j * num_lanes + k];
        if (x_norm > 0) {
            int x = (int)(x_norm * col_sample_w *
            outputWidth / 800.0f);
            int y = (int)((((float)row_anchor[17 - j])
            / 288.0f * outputHeight);
            lanePoints[pointIdx].x = x;
            lanePoints[pointIdx].y = y;
            pointIdx++;
        }
    }
}

// draw all lane points
if (pointIdx > 0) {
    cvDrawPoints(outputMapBase, outputWidth,
    outputHeight, lanePoints, pointIdx, 3, yellow);
}
```

TC-NN-APP 소스 수정

- tcnnapp 소스 빌드

```
$ cd ~/Work/ai-g-yocto
```

```
$ . poky/meta-topst/topst-build.sh -> 1번 머신 선택
```

```
$ bitbake -c compile -f tc-nn-app
```

```
$ cd tmp/work/cortexa53-telechips-linux/tc-nn-app/1.0.0-r0/build
```

```
$ scp -r tcnnapp root@192.168.0.100:/home/root/
```

TC-NN-APP 소스 수정

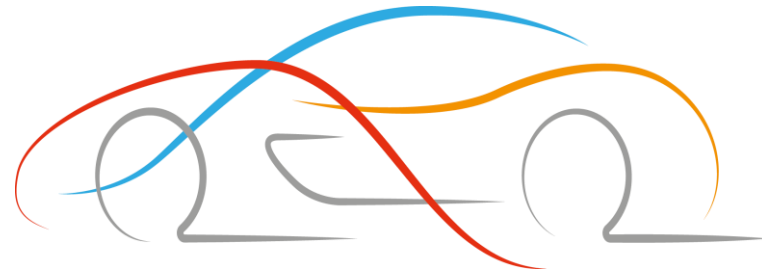
- AI-G 보드 확인

```
$ ls
```

```
tcnnapp
```

- 권한 부여

```
$ chmod 755 tcnnapp
```



Thank You